Training a Small Autoregressive LLM on Hacker News Headlines

Hugh Signoriello

1 Introduction

This report details the architectural and optimisation improvements made to an autoregressive transformer model trained on Hacker News headline text to predict the next token. The Lllama-based transformer acheives a minimum validation loss of around 1.223 on the hacker news dataset when trained on 7 epochs with a 10% validation split.

2 Changes Made

To progress beyond the baseline GPT-style transformer architecture, I implemented a LLaMA-style transformer architecture broadly based on Meta's LLaMA model family. This architecture was chosen due to its demonstrated empirical performance advantages over standard GPT baselines across numerous language benchmarks [1]. It also offers improved training efficiency, delivering competitive performance with substantially fewer parameters compared to GPT-3 and PaLM architectures [1]. This parameter efficiency is especially advantageous given the operational constraint of a 7-epoch training limit and local compute availability. The overall LLaMA-style architecture defined in `llama_model.py` has 22.97M parameters with 6 transformer layers, 8 attention heads, and an embedding dimension of $d_{\text{model}} = 512$. These parameters were found to be appropriate for the size of the Hacker News dataset, and small enough to allow for fast iteration but not large enough to cause overfitting. The overall architecture is outlined in Figure 1 below.

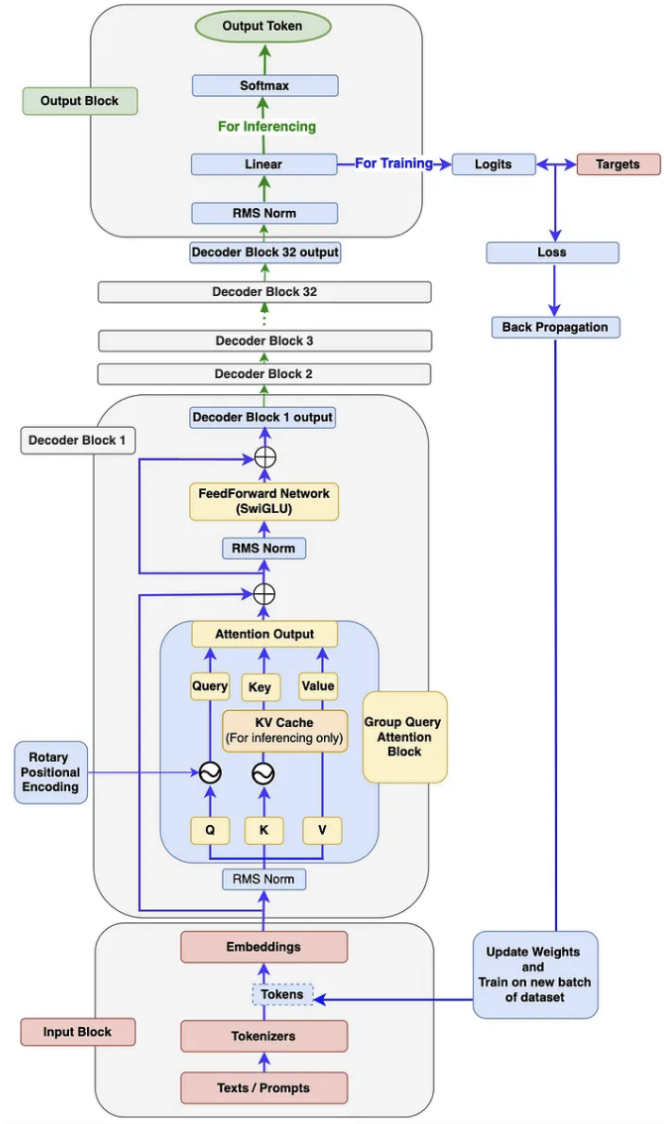


Figure 1: LLaMA-3 Architecture [6].

2.1 Tokenisation

The dataset loading, Byte Pair Encoding (BPE) tokenizer training, and data batching logic remain the same as baseline and this logic was moved to `data.py` to seperate concerns. The BPE tokenization algorithm which is used in the official llama implementation [1] and is trained on 100,000 Hacker News titles and has a vocabulary size of 8000 tokens. Decreasing vocabulary from 16k to 8k tokens size led to reduced validation loss. Also infrequent words might be broken down into common subword tokens allowing the model to better generalise to unseen text and may give the model more capacity for tranfomer layers to learn dataset terminology with the reduced token vocabulary size.

2.2 Root Mean Square Layer Normalisation (RMSNorm)

I replaced standard LayerNorm with Root Mean Square Layer Normalisation (RMSNorm). Unlike standard LayerNorm, RMSNorm using pyTorch's `nn.RMSNorm` and is favoured as it is computationally simpler and only requires a single pass making it more efficient than LayerNorm [5] which may imrprove training speed and stability of transformer training.

2.3 SwiGLU Feed-Forward Network

I replaced standard 2-layer GELU MLPs with Swish-Gated Linear Units (SwiGLU) [4], as integrated into the `LlamaMLP` block. The SwiGLU activation function behaves similarly to ReLU on positive inputs, but outputs small negative values for negative inputs [6] and Swish has been shown to outperform ReLU in many applications and it provides a smoother transition around 0, which may lead to faster training convergence. This is particulary advantageous considering the 7 epoch constraint.

$$\text{SwiGLU}(x) = (\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}}))\, W_{\text{down}} \tag{1}$$

To maintain the parameter budget relative to standard $4 \times d_{\text{model}}$ MLPs, the intermediate hidden dimension was scaled by $\frac{2}{3} \times 4d_{\text{model}} = \frac{8}{3}d_{\text{model}}$ following LLaMA specifications [1]:

2.4 Rotary Position Embeddings (RoPE)

The absolute positional embeddings were removed and replaced with rotary positional embeddings (RoPE) [1]. RoPE is a type of position encoding that maintains the order of tokens in sequences by incorporating both absolute position and relative position information among tokens [6]. It performs positional encoding by rotating query and key representations using a rotation matrix. RoPE is advantageous because it naturally incorporates relative position dependency into the self-attention formulation, providing flexibility across sequence lengths and decaying inter-token dependency with increasing relative distance [3] compared to learned absolute embeddings. As a result of helping the transfomer genralise better to longer sequences and learn positional context more effectively than traditional absolute position enbeddings, RoPE embeddigns may decrease the loss function. The implementation was adapted from: Hugging Face Transformers `modeling_llama.py`. In the `rotate_half` method, the vector is split down the middle. The first half represents all the x-coordinates, and the second half represents all the y-coordinates for which the rotation matrix is applied.

3 Training Process and Hyperparameter Selection

In the first few training runs I experimented with the training hyperpamaters to see how they affected validation loss, particularly on the GPT-style transformer architecture and then later experimented with these hyperparamaters on the Llama-based transformer training.

- **Total steps:** The transformer trains on 100,000 Hacker News titles over 7 epochs, which equals 2,247 total steps.
- **Dropout:** In earlier runs, specifically exp-05-llama I observed that validation loss began to increase around Epoch 5 with a minimum validation loss of 1.240 followed by increased validation loss while training loss continued to decrease, suggesting mild overfitting. To mitigate this and smooth the validation curve, a dropout rate of 0.05 was introduced so that the network will randomly deactivate 5% of its neurons in a given layer during each training iteration, allowing the model to better learn generalised patterns and mitigate overfitting.
- **Optimiser:** The suggested optimiser for the transfomer is AdamW [2]. The hyperparameters for AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 10^{-8}$ are selected following LLaMA's suggested training specifications [1]. A weight decay of $\lambda = 0.1$ is applied selectively to all 2D weight matrices [1]. Since LLaMA does not have bias terms in its linear layers, all 1D parameters correspond to RMSNorm scale vectors; these are explicitly excluded from weight decay ($\lambda = 0.0$) and are reflected in the `optim_groups` dictionary in `train.py`.

- **Scheduler:** While the original LLaMA models employ 2,000 warmup steps, this smaller-scale model is trained over fewer total steps. The warmup period was therefore scaled down proportionally to 200 steps or around 1% of the total training steps. Having a warmup prevents early training instability and divergence when training deep transformers with adaptive gradient optimisers like AdamW.

  Following warmup, a cosine annealing schedule decays the learning rate down to 10% of its peak value ($\eta_{\text{min}} = 0.1 \times \text{lr}$) [1]. The cosine decay helps smoothly curve the learning rate downwards following a cosine function until the end of training, this helps prevent the model from exiting a good local minimum when the learning rate is at its maximum.

- **Hardware:** I trained the transformer on Apple Silicon MPS on my M4 mac reduce runtime leading to a significant speedup around five-fold over CPU execution when running in Docker container.

4 Experimental Results

Exp ID	Architecture & Setup	LR	Params	Train Loss	Val Loss
Baseline	GPT-2, SGD, vocab-size 16k, block 128	6.0×10^{-3}	27.2M	8.179	1.754
exp-001-hp	GPT-2, SGD, vocab-size 8k, block 64	1.0×10^{-3}	23.0M	8.223	2.025
exp-002-hp	GPT-2, AdamW, vocab-size 16k, block 128, 0.1 dropout	6.0×10^{-4}	27.2M	5.802	1.291
exp-003-hp	GPT-2, AdamW, vocab-size 8k, block 32	8.0×10^{-4}	23.0M	4.480	1.270
exp-004-llama	LLaMA	8.0×10^{-4}	22.97M	4.110	1.243
exp-005-llama	LLaMA, Warmup + DecayWarmup	6.0×10^{-4}	22.97M	3.656	1.252
exp-006-llama	**LLaMA, Warmup + Decay, Dropout 0.05**	$\mathbf{6.0 \times 10^{-4}}$	**22.97M**	**4.035**	**1.223**

Table 1: Validation loss progression across training runs within the 7-epoch limit.

References

[1] Touvron, H., et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971.

[2] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.

[3] Su, J., et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864.

[4] Shazeer, N. (2020). *GLU Variants Improve Transformer*. arXiv preprint arXiv:2002.05202.

[5] Zhang, B., & Sennrich, R. (2019). *Root Mean Square Layer Normalization*. NeurIPS.

[6] Tamang, M. (2024). *Build Your Own LLaMA 3 Architecture from Scratch Using PyTorch*. Towards AI. https://pub.towardsai.net/build-your-own-llama-3-architecture-from-scratch-using-pytorch-2ce1ecaa901c